Aprendizaje no supervisado: segmentación de nubes de puntos LiDAR con k-means y DBSCAN

Juan José Cabrera y Arturo Gil

27 de octubre de 2026

1. Introducción

Hasta ahora, todo el aprendizaje que hemos hecho ha sido **supervisado**: disponíamos de ejemplos x **y de su respuesta correcta** y. El **aprendizaje no supervisado** trabaja únicamente con X: el algoritmo debe descubrir por sí mismo la estructura que hay en los datos. En este ámbito, la tarea más habitual y conocida se denomina **agrupamiento** (*clustering*): búsqueda de grupos de ejemplos parecidos entre sí.

En esta práctica agruparemos los datos de un sensor LiDAR. Este tipo de sensor está instalado en una gran cantidad de coches autónomos y muchos robots. ¿Para qué se instala este sensor?: un coche que circula necesita saber qué parte de lo que ve es **suelo** y qué **objetos** (coches, peatones, farolas, muros) tiene alrededor. En este contexto, resulta inviable etiquetar a mano cada punto de cada nube: un solo barrido tiene más de cien mil puntos y el sensor produce diez por segundo. ¿Hasta dónde podemos llegar **sin etiquetas**?

Las etiquetas sólo sirven para medir

Nuestros barridos tienen etiquetas puestas a mano, pero **los algoritmos que programaremos en esta práctica no las verán nunca**: usaremos técnicas que agrupan solamente utilizando las coordenadas cartesianas de los puntos o bien otras características de los puntos. Las etiquetas se usarán, *después*, para medir la precisión del proceso de *clustering*. En un problema real no existen estas etiquetas, y la única forma de juzgar un agrupamiento es **dibujarlo y observarlo**. Por esta razón, durante esta práctica usaremos la librería `open3d` para juzgar el resultado.

1.1. El sensor LiDAR

Un LiDAR (*Light Detection and Ranging*) emite pulsos de luz láser en diferentes direcciones. El sensor mide el **tiempo de vuelo** desde que la luz sale del sensor, impacta sobre un objeto y se refleja de vuelta sobre el sensor: como la luz viaja a una velocidad conocida ($3 \cdot 10^8$ m/s, en el vacío), el tiempo de vuelo permite deducir la distancia al obstáculo. Conociendo además la dirección en la que salió el pulso, cada eco es un **punto 3D**. Usaremos barridos reales del conjunto de datos **KITTI**[1], grabados en Karlsruhe con un Velodyne HDL-64E montado en el techo de un coche. Las características de este sensor son:

- **64 láseres** apilados verticalmente que giran 360° diez veces por segundo. Cada láser describe un círculo sobre el suelo: por eso las nubes se ven como **anillos** concéntricos.
- La densidad de puntos **decae con la distancia**: los objetos cercanos tienen miles de puntos y los lejanos, unos pocos.
- Ejes del sensor: x hacia **delante** del coche, y hacia la **izquierda** y z hacia **arriba**. El sensor va a 1,73 m del suelo, así que el suelo está en torno a $z \approx -1{,}7$ m.

[1] A. Geiger, P. Lenz y R. Urtasun, *Are we ready for autonomous driving? The KITTI vision benchmark suite*, CVPR, 2012. Licencia CC BY-NC-SA 3.0.

- Cada punto guarda también la **intensidad** del eco, entre 0 y 1: las superficies brillantes (matrículas, señales, marcas viales) devuelven más intensidad de luz.

Un barrido se guarda en un fichero `.bin` que no es más que una ristra de números `float32`, cuatro por cada punto: x, y, z, intensidad. Sus etiquetas son las de **SemanticKITTI**[2]: un fichero `.label` con un entero por punto que dice su **clase** (calzada, acera, coche, edificio, vegetación…) y, en los objetos, su **instancia** (qué coche concreto es).

1.2. Materiales

Trabajaremos en el directorio `practice_unsupervised_learning_lidar` del repositorio Intro2AI (https://github.com/4rtur1t0/Intro2AI). Instale `open3d` si lo necesita: `pip install open3d`. El directorio contiene:

- `data/`: dos barridos con sus etiquetas. El principal, `kitti_00_001560`, es un cruce de calles de una zona residencial, con coches aparcados, edificios y árboles. El otro, `kitti_08_003880`, es para el apartado opcional.
- `lidar/nube.py`: lectura, recorte, colores y visualización **(ejercicio 1)**.
- `lidar/kmeans.py`: k-means desde cero **(ejercicio 2)**.
- `lidar/caracteristicas.py`: características geométricas de cada punto, ya programadas **(ejercicio 3)**.
- `lidar/suelo.py`: qué puntos son suelo según las etiquetas y las medidas de acierto, ya programado.
- `1_visualizar_main.py`, …, `4_dbscan_main.py`: un programa por ejercicio.

Los programas se ejecutan desde dentro del directorio (`python 1_visualizar_main.py`). Las partes que deben completarse están marcadas con `EJERCICIO: SE DEBE COMPLETAR EL CÓDIGO POR EL ESTUDIANTE` y lanzan `NotImplementedError` mientras no se completen. Cada ventana de `open3d` detiene el programa hasta que se cierra. Ratón: el botón izquierdo gira, la rueda acerca y `Ctrl`+botón izquierdo desplaza; las teclas `+` y `-` cambian el tamaño de los puntos.

Antes de comenzar

Asegúrate de que tu editor de python está configurado correctamente:

- El intérprete de python debe apuntar al entorno virtual de las prácticas `venvAI`.
- Usa el gestor de `git` para obtener la última versión del repositorio de prácticas.

2. Leer y ver un barrido

Lea la nube con

```
nube = np.fromfile(ruta, dtype=np.float32)
nube = np.reshape(nube, [-1, 4])
```

`np.fromfile` devuelve todos los números seguidos en un vector; `reshape` con -1 los coloca en filas de cuatro y calcula él solo cuántas filas salen.

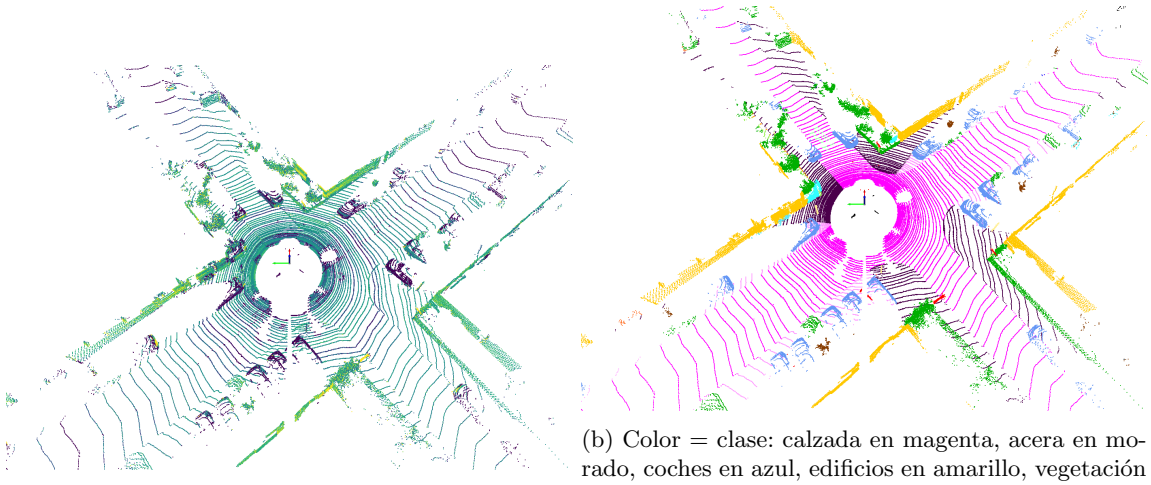
[2] J. Behley et al., *SemanticKITTI: A Dataset for Semantic Scene Understanding of LiDAR Sequences*, ICCV, 2019. Licencia CC BY-NC-SA 4.0.

Ejercicio 1

Complete `cargar_nube` y `a_open3d` en `lidar/nube.py` y los huecos de `1_visualizar_main.py`:

1. Calcule el **número de puntos** del barrido. Debe obtener **122.485** puntos.
2. Visualice la nube en `open3d` **coloreada por la intensidad** (columna 3). La función `colores_por_valor` ya convierte un valor por punto en un color.

El programa termina mostrando la nube **coloreada por clase** con las etiquetas de Semantic-KITTI (figura 1) y una tabla con las clases presentes: calzada (33,7 %), edificio (17,5 %), acera (15,4 %), vegetación (12,5 %), coche (10,5 %)...



(a) Color = intensidad.

(b) Color = clase: calzada en magenta, acera en morado, coches en azul, edificios en amarillo, vegetación en verde.

Figura 1: El barrido 1560 de la secuencia 00, visto desde arriba con el coche avanzando hacia la parte superior de la imagen. Los ejes en el centro marcan el sensor (rojo x, verde y, azul z).

3. k-means desde cero

El algoritmo *k-means* plantea la agrupación de nuestro conjunto de datos en k conjuntos. Para ello, el algoritmo genera k **centroides** aleatorios. Cada punto del conjunto de datos se asocia a un centroide (al más cercano, dada una función de distancia). Seguidamente, se recalculan los centroides (como el valor medio de los puntos que tiene asociados). En definitiva, el algoritmo repite los siguientes pasos hasta que se estabiliza:

1. **Asignación**: cada punto se asigna al centroide más cercano.
2. **Actualización**: cada centroide se mueve a la **media** de los puntos que se le han asignado.

En cada iteración, el algoritmo disminuye la **inercia** total J, definida como la suma de las distancias al cuadrado de cada punto x_i a su centroide $c_{j(i)}$:

$$J = \sum_{i=1}^{N} \|x_i - c_{j(i)}\|^2.$$

Ejercicio 2

Se desea que el/la estudiante sea capaz de programar el algoritmo `k-means` usando la librería numpy.
EJERCICIO

- Escriba una **implementación completa del algoritmo** *k-means* usando únicamente python y numpy: complete el código de `asignar`, `actualizar`, `inercia` y `kmeans` en `lidar/kmeans.py` usando `numpy`. No recorra los puntos uno a uno en un bucle: con cien mil puntos sería muy lento; recorra los k centroides y opere con todos los puntos a la vez.
- Use la clase `KMeans` de scikit-learn para agupar los datos. Para ello, ejecute `2_kmeans_main.py`. Agrupa las coordenadas (x, y, z) del recorte de 40 m × 40 m alrededor del sensor (109.994 puntos) en $k = 4$ grupos y compare su *k-means* con el de `scikit-learn` partiendo de los **mismos centroides iniciales**.
- Si su implementación es correcta, los resultados de ambos algoritmos deben dar, como resultado, la misma inercia (**4.136.190,7**), el mismo número de iteraciones (37) y el 100 % de etiquetas iguales. ¿Qué forma tienen los cuatro grupos que se ven en `open3d`? ¿Separan objetos o simplemente reparten el espacio? Pruebe a cambiar k=5, 6, 7 . . . . ¿Qué observa?

Hemos podido implementar el algoritmo k-means con un resultado igual al muy probado algoritmo de la librería Scikit-learn. Sin embargo, **la agrupación de los puntos no nos proporciona ninguna información de provecho**: simplemente agrupa los puntos por cercanía, mientras minimiza la inercia total del conjunto. **Así pues, podemos asegurar que la posición del punto no es una característica de interés para poder agrupar los puntos en entidades con significado.**

4. Suelo frente al resto: la importancia de las características

En este apartado nos vamos a enfrentar a una tarea más sencilla: separar la nube recortada en **dos** grupos, suelo y todo lo demás. Según las etiquetas proporcionadas por KITTI, el 57,1 % de sus puntos son suelo (calzada, acera, aparcamiento, terreno. . .). A continuación, volveremos a ejecutar el algoritmo k-means de la misma manera, pero cambiaremos la información de cada punto. El algoritmo será siempre el mismo, k-means con $k = 2$, pero a cada punto le asignaremos otra información.

Cómo se evalúa: matriz de confusión y métricas

El algoritmo *k-means* es estrictamente un algoritmo de agrupación (*clustering*): no es un algoritmo de clasificación. Así pues, el resultado del algoritmo es un conjunto de *datapoints* agrupados, pero por defecto ninguno de esos grupos se asocia a una etiqueta de suelo u otros. En el caso que nos ocupa, y con la intención de poder evaluar la calidad de la agrupación de los datos, hacemos una presunción importante: se toma la correspondencia grupo → clase que más puntos acierta, como se hace siempre al comparar un agrupamiento con etiquetas. Con el **suelo como clase positiva**, cada punto cae en una casilla de la **matriz de confusión**: **TP** (suelo predicho como suelo), **FN** (suelo predicho como objeto), **FP** (objeto predicho como suelo) y **TN** (objeto predicho como objeto). De ella salen las métricas estándar:

$$\text{accuracy} = \frac{TP+TN}{N}, \qquad \text{precision} = \frac{TP}{TP+FP}, \qquad \text{recall} = \frac{TP}{TP+FN},$$
$$\text{F1-score} = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2TP}{2TP+FP+FN}, \qquad \text{IoU} = \frac{TP}{TP+FP+FN}.$$

La **precision** dice qué parte de lo que se predice suelo lo es de verdad; el **recall**, qué parte del suelo real se encuentra. El **F1-score** los resume con su media armónica, que solo es alta si *ambos* lo son. El **IoU** (intersección sobre unión, o índice de Jaccard) es la métrica oficial de SemanticKITTI: el tamaño de lo que coincide entre predicción y etiqueta dividido por el tamaño total de las dos. Como en la práctica 3, la **accuracy** sola puede resultar engañosa cuando una clase es mucho más frecuente que la otra. Como recordatorio, damos una definición formal de los anteriores parámetros y lo aplicamos al caso que nos ocupa:

- **Accuracy**: la proporción de todas las predicciones correctas sobre las clases (en nuestro caso, sobre suelo y otros). **Útil**: cuando el dataset está equilibrado y las predicciones sobre ambas clases también son correctas. **Limitación**: en el caso en el que el dataset está desequilibrado.
- **Precision**: la cantidad de predicciones positivas correctas (TP) sobre el total de predicciones positivas (TP+FP). En nuestro caso, este parámetro mide la cantidad de puntos que se han clasificado como suelo y eran realmente suelo (TP) sobre el total de puntos que se han clasificado como suelo (TP+FP). **Útil**: cuando la aparición de FP es costosa. En el caso que nos ocupa, si nuestro clasificador encuentra un FP, ¿qué ocurre?: hay un punto que se ha clasificado como "suelo" que intentará ser transitado por nuestro coche autónomo, cuando realmente "no es suelo". En definitiva, **la precisión indica cómo de fiables son los puntos que se han clasificado como suelo**.
- **Recall**: se calcula como el número de casos positivos reales (TP) que logró detectar el modelo sobre el total de casos positivos que hay en el dataset (TP+FN). En nuestro caso, mide la capacidad del clasificador para encontrar elementos de tipo "suelo" de entre el total de puntos de suelo existentes en el dataset.
- **IoU**: mide la ratio entre los TP sobre el total de positivos en el *ground truth*. En esencia es un parámetro que considera la capacidad del modelo para generar TP y lo pondera con la cantidad de FP y FN que también genera el modelo.
- **F1-score**: es la media harmónica entre precision y recall. Se utiliza para poder simplificar el problema y utilizar un único parámetro para comparar diferentes métodos de clasificación.

Características geométricas (ya calculadas)

Para cada punto, la función `caracteristicas_geometricas` toma sus vecinos a menos de 0,5 m y calcula la matriz de covarianza 3×3 de sus coordenadas. Sus autovectores definen las direcciones en que se extiende el vecindario (es un PCA local) y sus autovalores $\lambda_1 \geq \lambda_2 \geq \lambda_3$, indican amplitud en cada una de esas direcciones. La **normal ñ** de la superficie es la dirección en la que el vecindario varía *menos*, esto es, la dirección asociada a λ_3. De ahí salen dos características numéricas por cada punto de cada nube:

- **Verticalidad** $= 1 - |n_z|$: 0 en superficies horizontales (calzada, techo de un coche) y 1 en verticales (paredes, laterales de los coches, troncos).
- **Curvatura** $= \lambda_3/(\lambda_1 + \lambda_2 + \lambda_3)$: 0 en superficies lisas y hasta 1/3 en vecindarios desordenados (vegetación, esquinas).

Antes de agrupar, cada característica se **estandariza** (se le resta su media y se divide por su desviación típica). De esta manera, eliminamos la influencia de las unidades y problemas de convergencia debido a la función de distancia.

Ejercicio 3

Deseamos aplicar el algoritmo k-means a los puntos de una nube. Sin embargo, ahora vamos a intentar agrupar estos puntos usando las características geométricas que hemos descrito anteriormente:

EJERCICIO

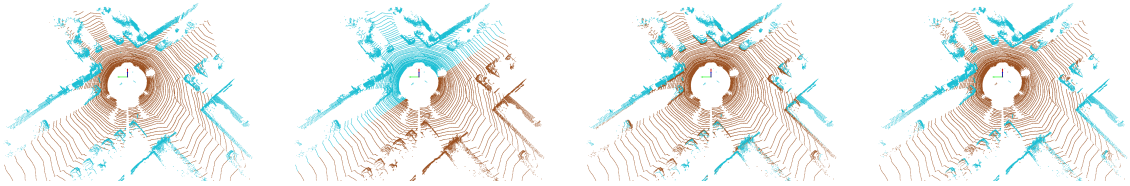
Complete los huecos de `3_suelo_main.py` y ejecútelo.

- En la primera parte del programa, se muestra el suelo con `open3d` según las etiquetas conocidas de KITTI (figura 2a). A continuación, se evalúa la clasificación usando una **referencia trivial**. Con esta clasificación trivial (que dice que todo es suelo): accuracy 57,1 %, precision 57,1 %, recall 100 % y F1-score 72,7 %. **Nótese que este resultado es importante**, pues establece un límite inferior de los resultados: si hacemos una clasificación arbitraria, el F1-score es del 70 % aproximadamente, debemos superar este resultado con cualquier otra clasificación que hagamos.
- Seguidamente, en el programa podemos cambiar los atributos usados como entrada a *k-means*. Para cada elección de características el programa imprime la matriz de confusión y las cinco métricas (tabla 1).

  a) Atributos (x, y, z). Debe obtener una accuracy del **51,8 %**, *peor* que la referencia trivial, y un F1-score del 50,2 %. Observe el resultado en `open3d`: ¿por dónde corta k-means la nube? ¿Por qué, si x e y varían en 40 m y z apenas en unos metros?

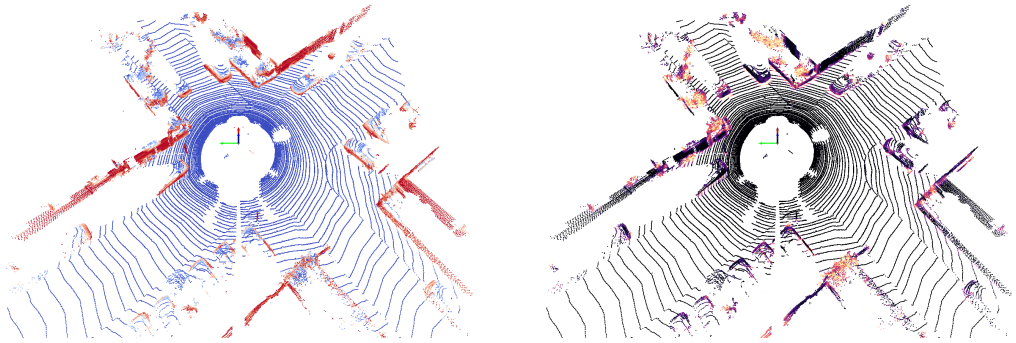
  b) Sólo el atributo z (cuidado: la matriz debe ser $N \times 1$, no un vector). Debe obtener un **recall del 100 %** pero una **precision del 79,8 %**: hay 15.917 falsos positivos. Los centroides quedan en $z = -1{,}67$ y $z = -0{,}20$ m. Calcule la **frontera** entre los dos grupos, la altura a partir de la cual un punto pasa de un grupo al otro ($-0{,}93$ m). ¿Qué son esos falsos positivos, qué parte de los coches y de los muros acaba llamando "suelo" k-means? ¿Por qué?

  c) Mire primero en `open3d` la nube coloreada por verticalidad y por curvatura (figura 3): ¿qué sale en rojo?, ¿dónde es alta la curvatura? Después construya la matriz con z, verticalidad y curvatura. Deberá obtener una precision del **95,9 %** y un recall del **98,6 %** (F1-score 97,2 %, IoU 94,6 %). ¿Qué ha arreglado la verticalidad respecto al apartado b)? Por último, comente la línea que estandariza los datos y vuelva a ejecutar: la precision baja al 88,1 % (F1-score 93,7 %). ¿Por qué? (Pista: la desviación típica de la z es 0,75 m y la de la curvatura, 0,05.)



(a) Etiquetas. (b) (x, y, z). (c) z. (d) z + geometría.

Figura 2: Suelo (marrón) frente al resto (azul) en el recorte de 40 m × 40 m: (a) según las etiquetas; (b)–(d) según k-means con $k = 2$ y cada elección de atributos.



(a) Verticalidad: azul horizontal, rojo vertical. (b) Curvatura: oscuro liso, claro desordenado.

Figura 3: Las dos características geométricas del recorte.

Moraleja

El algoritmo era el mismo en los tres casos. k-means no busca "suelo" ni "objetos": reparte los puntos según su distancia a unos centroides *en el espacio de atributos que le damos*. **Elegir bien las características importa más que el propio algoritmo.**

5. Objetos con DBSCAN

Para encontrar objetos, k-means tiene dos problemas: 1) hay que decirle **cuántos** grupos, y 2) los grupos generados son siempre "redondos" alrededor de un centroide. El algoritmo **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*) agrupa por **densidad** con dos parámetros: un radio `eps` y un número mínimo de vecinos `min_samples`.

- Un punto es **núcleo** si tiene al menos `min_samples` puntos (contándose a sí mismo) a una distancia menor de `eps`.
- Dos núcleos a una distancia menor de `eps` se fusionan en el mismo grupo; así, un grupo crece encadenando núcleos y puede tener **cualquier forma**.
- Un punto que no es núcleo pero está una distancia menor de `eps` de un punto que es **borde**, se une a su grupo. El resto es **ruido** (etiqueta -1).

El número de grupos resultante lo decide el algoritmo. En el dataset de KITTI, en la práctica el suelo conecta todos los objetos entre sí. Para evitar este efecto indeseado, primero se elimina el suelo y se agrupa en **vista cenital**, sólo con las coordenadas (x, y). Aquí el suelo se quita con las etiquetas, para estudiar DBSCAN por separado; en un coche real se quitaría con un filtro geométrico (un plano ajustado con RANSAC, el filtro CSF…) o con un agrupamiento como el del ejercicio 4. También se quitan los puntos a más de unos 2,5 m de altura (copas de árboles, aleros), que vistos desde arriba unirían lo que tienen debajo. Para medir el resultado, las etiquetas de instancia dicen qué puntos son

Atributos de k-means ($k = 2$)	Accuracy	Precision	Recall	F1-score	IoU
Referencia trivial: todo es suelo	57,1 %	57,1 %	100,0 %	72,7 %	57,1 %
(x, y, z)	51,8 %	61,4 %	42,5 %	50,2 %	33,5 %
z	85,5 %	79,8 %	100,0 %	88,7 %	79,8 %
z + verticalidad + curvatura	96,8 %	95,9 %	98,6 %	97,2 %	94,6 %

Cuadro 1: Segmentación del suelo (clase positiva) con k-means, $k = 2$, en el barrido 00/001560.

de cada coche. Se usa el criterio estándar de la segmentación panóptica[3]: un coche es **detectado** si algún grupo tiene con él un IoU $> 0{,}5$, donde

$$\text{IoU}(\text{coche}, \text{grupo}) = \frac{|\text{coche} \cap \text{grupo}|}{|\text{coche} \cup \text{grupo}|}$$

Con ese umbral cada coche solo puede emparejarse con un grupo, y, como en SemanticKITTI, se ignoran los coches con menos de 50 puntos. El programa da el **recall** (los coches detectados sobre el total de coches existentes) y el **IoU medio** de los detectados, que indica cuánto se parece cada grupo a su coche.

Ejercicio 4

Deseamos observar el resultado de DBSCAN al variar algunos de sus parámetros.
EJERCICIO
Complete `agrupar_dbscan` en `4_dbscan_main.py` usando `sklearn.cluster.DBSCAN` y ejecute el programa.

a) Con `eps` = 0,5 m y `min_samples` = 8 sobre los 46 769 puntos de objeto debe obtener **45 grupos y 75 puntos de ruido**, con **15 de los 16 coches** detectados (recall 93,8 %) y un IoU medio de **0,97**. Busque en `open3d` el coche que falla, a unos 20 m del sensor: ¿con qué se ha juntado?

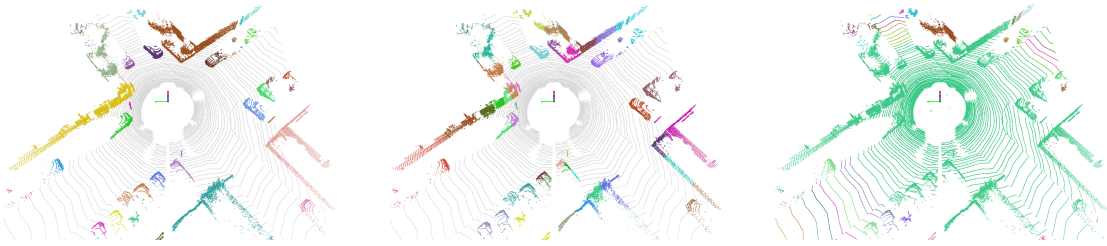
b) El programa repite DBSCAN con varios valores `eps`: debe ver 98, 66, 45, 25 y 6 grupos para 0,2, 0,3, 0,5, 1 y 2 m, con un recall de coches del 87,5, 100, 93,8, 56,2 y 6,2 %. Explique qué ocurre con los objetos cuando `eps` es demasiado pequeño y cuando es demasiado grande. Con 0,3 m se detectan todos los coches; ¿por qué aun así puede no ser la mejor elección? (Pista: la métrica solo mira los coches; fíjese en el número de grupos y de puntos de ruido.)

c) El programa usa *k-means* (scikit-learn) con $k = 45$, el número de grupos de DBSCAN, detecta solo **10 de 16** coches (recall 62,5 %), con un IoU medio de **0,80**. Compare en `open3d` con el apartado a): ¿qué hace k-means con un muro largo? ¿Y con dos coches aparcados muy juntos?

d) DBSCAN **sin quitar el suelo**, en (x, y, z). ¿Qué porcentaje de los puntos acaba en el grupo mayor (90,9 %)? ¿Por qué?

Opcional: cambie `BARRIDO` por `data/kitti_08_003880` en los ejercicios 3 y 4. En el ejercicio 3, el F1-score pasa del 71,8 % con la z sola al 88,7 % con las características geométricas (la referencia trivial da 55,9 %). En el 4, DBSCAN detecta 9 de 11 coches (IoU medio 0,90) y k-means, 5 (IoU medio 0,78).

[3] A. Kirillov et al., *Panoptic Segmentation*, CVPR, 2019. Es el criterio del *benchmark* panóptico de SemanticKITTI.



(a) DBSCAN, 45 grupos. (b) k-means, $k = 45$. (c) DBSCAN con el suelo.

Figura 4: Un color por grupo. En (a) y (b) el suelo, que no se agrupa, aparece en gris claro; en (a) el ruido aparece en gris.

Para terminar

k-means agrupa los puntos por **cómo son** (sus características: altura, verticalidad, curvatura); DBSCAN, por **dónde están** (qué puntos se tocan con cuáles). Juntos forman un sistema de percepción completo: primero se separa el suelo de lo demás y después lo demás se trocea en objetos.